

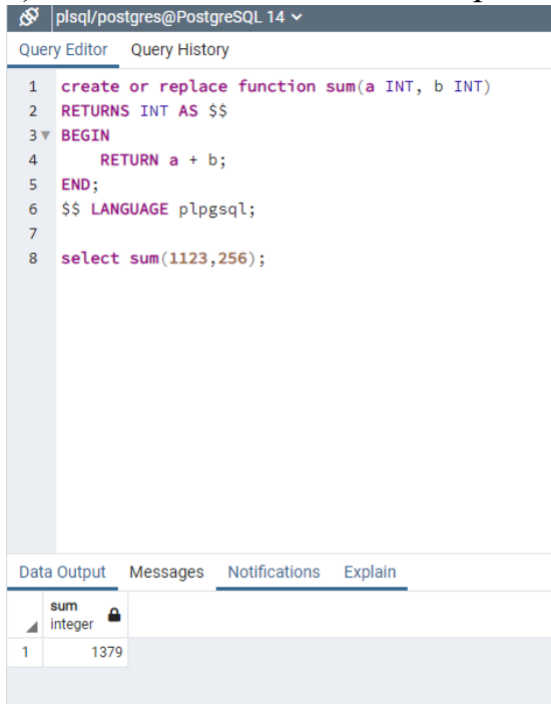
PL-SQL Lab 1

Name : Devananda T C

Roll No : AM.SC.U4AIE23111

1. Do all the questions discussed in the class

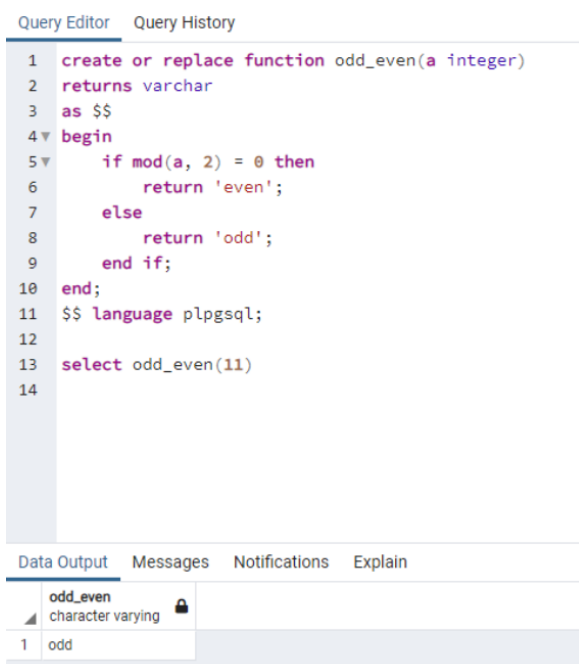
a) Write a function which accepts 2 integer values and return their sum.



```
1 create or replace function sum(a INT, b INT)
2 RETURNS INT AS $$
3 BEGIN
4     RETURN a + b;
5 END;
6 $$ LANGUAGE plpgsql;
7
8 select sum(1123,256);
```

sum
integer
1 1379

b) Write a function which accepts an integer value and returns if it is odd or even



```
1 create or replace function odd_even(a integer)
2 returns varchar
3 as $$
4 begin
5     if mod(a, 2) = 0 then
6         return 'even';
7     else
8         return 'odd';
9     end if;
10 end;
11 $$ language plpgsql;
12
13 select odd_even(11)
14
```

odd_even
character varying
1 odd

- c) Write a function which accepts employee number and returns salary of that employee

```
10 create or replace function emp_sal(empno Emp.eno%type)
11 returns Emp.sal%type as
12 $$
13     Declare
14         salary emp.sal%type;
15     Begin
16         select sal into salary from emp
17         where eno = empno;
18         return salary;
19     End;
20 $$
21 Language 'plpgsql';
22
23 select emp_sal(103) as salary;
24
25
26
```

Data Output Messages Notifications

SQL

	salary integer
1	60000

- d) Write a function which accepts a dept no. and finds the number pf employees in that department

```
5 create or replace function emp_count(dept_no Emp.dno%type)
6 returns integer as
7 $$
8     Declare
9         cnt integer;
10    Begin
11        select count(*) into cnt from Emp
12        where dno = dept_no;
13
14        return cnt;
15    End;
16 $$
17 Language 'plpgsql';
18
19 select emp_count(10) as total_employees;
20
```

Data Output Messages Notifications

SQL

	total_employees integer
1	1

- e) Write a function which takes an employee number and give that employee 10% hike in salary , if average salary of the corresponding department is greater than employees current salary.

```
Query Query History
1 create or replace function emp_hike(emp_no emp.eno%type)
2 returns emp.sal%type as
3 $$
4 declare
5 salary emp.sal%type;
6 begin
7 select sal into salary from emp where eno = emp_no;
8 if salary < (select avg(sal) from emp
9 where dno = (select dno from emp where eno = emp_no)) then
10 update emp
11 set sal = sal + (sal * 0.10)
12 where eno = emp_no;
13 select sal into salary from emp where eno = emp_no;
14 end if;
15 return salary;
16 end;
17 $$
18 language 'plpgsql';
19
20 select emp_hike(2) as updated_salary;
21
```

Data Output Messages Notifications

	updated_salary integer
1	[null]

- f) Write a function which accepts an employee name and returns the department name of that employee.

```
10 create or replace function get_deptname(emp_name emp.ename%type)
11 returns dept.dname%type as
12 $$
13 declare
14 deptname dept.dname%type;
15 begin
16 select dname into deptname from dept
17 where dno = (select dno from emp where ename = emp_name);
18 return deptname;
19 end;
20 $$
21 language plpgsql;
22
23 select get_deptname('Arun') as department_name;
24
```

Data Output Messages Notifications

	department_name character varying
1	HR

2. Consider the following schema .

Emp(eno, ename, sal)

EMP_PROJ(eno, pno, prjt_hrs).

- a) Write a function ProjectLoad that returns the total project working hours for the given eno.

```
create or replace function projectload(emp_no emp.eno%type)
    returns integer as
$$
    declare
        total_hrs integer;
    begin
        select sum(prjt_hrs) into total_hrs
        from emp_proj
        where eno = emp_no;

        return total_hrs;
    end;
$$
language plpgsql;

select projectload(101) as total_hours;
```

Output Messages Notifications



total_hours
integer
8

- b) Write a PL/SQL code to update the salary of an employee if the employee earn less than the average salary. New salary is current sal + difference between current sal and average salary.

```
2 create or replace function update_sal(emp_no emp.eno%type)
3     returns varchar as
4 $$
5     Declare
6         avg_sal    numeric;
7         curr_sal   numeric;
8         new_sal    numeric;
9
10    Begin
11        select sal into curr_sal from emp where eno = emp_no;
12        select avg(sal) into avg_sal from emp;
13
14        if curr_sal < avg_sal then
15            update emp
16                set sal = sal + (avg_sal - curr_sal)
17                where eno = emp_no;
18        end if;
19        select sal into new_sal from emp where eno = emp_no;
20        return new_sal;
21
22    end;
23 $$
24 Language 'plpgsql';
25
26 select update_sal(3);
27
```

Data Output Messages Notifications



update_sal
character varying
1 [null]

2. Consider the following tables

Item(ino, iname, unit_price)

Transaction(tr_no,ino,qty)

Write a function to accept an item no from the user. If transaction has been made for less than 2 times for that item(check from transaction table), delete the item from the Item table.

```
1 create table item(ino int primary key,
2   iname varchar(50),
3   unit_price numeric);
4 create table transactions(tr_no int,
5   ino int,qty int);
6 insert into item values
7   (101, 'Pen', 10),
8   (102, 'Book', 50),
9   (103, 'Bag', 300),
10  (104, 'Pencil', 5);
11 insert into transactions values
12   (1, 101, 2),
13   (2, 101, 3),
14   (3, 102, 1),
15   (4, 103, 4),
16   (5, 103, 2);
17 create or replace function delete_item_if_less(emp_ino item.ino%type)
18 returns varchar as
19 $$
20 declare
21   trans_count numeric;
22 begin
23   select count(*) into trans_count
24   from transactions
25   where ino = emp_ino;
26
27   if trans_count < 2 then
28     delete from item
29     where ino = emp_ino;
30   end if;
31
32   return 'Done';
33 end;
34 $$
35 language 'plpgsql';
37 select delete_item_if_less(101);
38
```

Data Output Messages Notifications



delete_item_if_less
character varying

1 Done

3. Consider a Bank database which includes the following tables.

ACCOUNTS(ac_no,br_no, cust_no,ac_type,bal)

BRANCHES(br_no, br_name, loc)

CUSTOMER(cno, cname, c_type)

- b) Write a function called CloseBranch that takes two arguments (the branch to be closed and the branch to take over the accounts) and transfers all accounts at the closing branch to the new branch and removes the closing branch.

```
Query Query History
1  --4(b)--
2  create or replace function CloseBranch(close_br branches.br_no%type,
3                                     new_br branches.br_no%type)
4  returns varchar as
5  $$
6  declare
7      acc_count int;
8  begin
9      update accounts
10     set br_no = new_br
11     where br_no = close_br;
12
13     delete from branches
14     where br_no = close_br;
15     return 'Done';
16 end;
17 $$
18 language plpgsql;
19
20 select CloseBranch(1, 2);
21
```

Data Output Messages Notifications

closebranch
character varying

1	Done
---	------

- c) Write a function that implements a "safe" withdrawal operation, that only permits a withdraw if there sufficient funds in the account to cover it.

```
Query Query History
1  --Safe Withdraw--
2  create or replace function safe_withdraw(accno accounts.ac_no%type,
3      amt numeric)
4  returns varchar as
5  $$
6  declare
7      curr_bal numeric;
8  begin
9      select bal into curr_bal
10     from accounts
11     where ac_no = accno;
12
13     if curr_bal >= amt then
14         update accounts
15         set bal = curr_bal - amt
16         where ac_no = accno;
17
18         return 'Success';
19     else
20         return 'Insufficient Funds';
21     end if;
22
23 end;
24 $$
25 language plpgsql;
26
27 select safe_withdraw(1001, 20000);
28
```

Data Output Messages Notifications

safe_withdraw
character varying

1	Success
---	---------